AI Engineer Technical Challenge – The Revenue Leakage Agent

Duration

1.5 hours (core demo)

Agenda of the meeting

20 min intro

1.5h offline agent building

20 min presenting the demo and answering the questions

Please prepare the API before the call

Context

This challenge simulates an AI “financial detective” that investigates **revenue leakage** and can **propose and apply fixes** in a sandbox.
All data is provided in JSON — no parsing required.

Key Concepts

Term	Meaning	Example Use
Credit Memo	A negative invoice that reduces what the customer owes — used when overbilled or a pricing error occurred.	Overbilled €25 000 → $27 000 → issue credit memo for $2 000 USD

Plan Amendment	An update to the contract/billing plan (total, cadence, entitlements) when the agreement itself changes.	Upgrade plan from $90 000 → $100 000 or add “Premium Support”
Make-Good Invoice	A new invoice to recover missed or underbilled revenue.	Missing September billing → invoice $8 000 USD

Objective

Build a **stateful AI agent** that can engage in conversations, answer questions, and perform actions. The agent should:

1. **Investigate** anomalies between billing plans and invoices
2. **Propose** corrective actions (make-good invoice, credit memo, or plan amendment)
3. **Apply** those actions to a writable **sandbox** with human approval
4. **Explain** what it did and why, citing evidence or calculations
5. **Answer questions** about billing plans, invoices, and revenue leakage issues
6. **Maintain context** across the conversation to enable follow-up questions and iterative investigation

The agent should be **conversational and stateful**, not a fixed workflow or pipeline. It should be able to have natural dialogues with users, remember previous interactions, and adapt its approach based on the conversation flow.

Example Conversation

Here's an example of how a user might interact with the agent:

User: Can you check if there are any revenue leakage issues with plan P-12345?

Agent: I'll investigate plan P-12345 for you. Let me load the plan details and check the invoices...
[Agent uses load_plan("P-12345") and query_invoices(...)]
I found a discrepancy! Plan P-12345 should bill $10,000 monthly, but I only see invoices for September and November. October is missing. This represents $10,000 in missed revenue.

User: What currency is that plan in?

Agent: Plan P-12345 is in USD. The expected monthly amount is $10,000 USD.

User: Can you create a make-good invoice for the missing October billing?

Agent: I'll propose a make-good invoice for $10,000 USD for the missing October 2024 billing period.
[Agent uses propose_make_good_invoice("P-12345", 10000, "Missing October 2024 billing")]
Proposal created. Would you like me to apply this to the sandbox?

User: Yes, apply it.

Agent: [Agent uses apply(...)] Make-good invoice has been applied to the sandbox. The invoice ID is INV-MG-001.

User: What about the other months? Are they correct?

Agent: Let me check the other invoices for this plan... September shows $10,000 USD which matches the plan. November also shows $10,000 USD. Both are correct. The only issue was the missing October invoice, which we've now addressed.

Notice how the agent: - Maintains context about plan P-12345 throughout the conversation - Answers follow-up questions that reference previous context ("What currency is that plan in?") - Understands references to previous actions ("What about the other months?") - Adapts its responses based on the conversation flow

---

Data Overview (`/data/`)

File	Description
`billing_plans.json`	Expected billing plans (contracts)
`invoices.json`	Issued invoices / bills
`credit_memos.json`	Existing credit memos

`exchange_rates.json`	FX rates
`/sandbox/*.json`	Writable ledgers for applied actions

Available Tools

Tool	Purpose
`load_plan(plan_id)`	Read plan details
`query_invoices(filters)`	Filter invoices by plan, customer, date
`fx_convert(amount, from_ccy, to_ccy, on_date)`	Currency conversion
`propose_make_good_invoice(plan_id, amount, reason)`	Draft new invoice
`propose_credit_memo(invoice_id, amount, reason)`	Draft credit memo
`propose_plan_amendment(plan_id, change_set)`	Draft billing plan update

`apply(action_draft)`	Write to `/sandbox` after confirmation
`rollback(action_id)`	Undo last applied action

UI Requirements

- **Chat interface only** (NextJS / React / Streamlit / ...)
- Users interact with the agent through natural conversation
- The agent can display findings, proposals, and request approval for actions within the chat
- Optional: reasoning trace & audit-log viewer (sandbox/audit_log.json)

AI Tool Usage Tracking

Important: Document all AI tool prompts used during this challenge in `candidate_prompts.md`. This includes: - ChatGPT, Claude, GitHub Copilot, Cursor, or any other AI tools - Exact prompts used and their context - Results and follow-up iterations - Reflection on tool effectiveness

This documentation is part of the evaluation process and helps assess AI tool utilization skills.

Submission

Submit your solution via private GitHub repository:

1. **Create a new private repository** on your personal GitHub account
2. **Copy the challenge files** to your new repository
3. **Implement your solution** in the repository
4. **Add the reviewer** as a collaborator with read access
5. **Submit the repository URL** to the reviewer